

SESIÓN

14

Escenarios transaccionales en ASP.NET Core

UNIDAD DE APRENDIZAJE 4

- Desarrollar aplicaciones web que manejen escenarios transaccionales, tipo carrito de compras o registro de tablas maestro–detalle como facturas u órdenes de compra.
- Generar reportes Excel desde aplicaciones ASP.NET Core con EF Core 10 y la librería EPPlus.

/ INTRODUCCIÓN

- En las aplicaciones web también manejamos escenarios transaccionales, como el registro de tablas con **cabecera y detalle**. El caso clásico es el “**carrito de compra**”: el usuario agrega artículos, cambia cantidades o los quita, y recién al final confirma o cancela toda la operación.
- También es frecuente que la aplicación permita **descargar archivos Excel** contruidos en el servidor con **EPPlus**, que ya usamos en la primera parte del curso.

En la sesión de hoy aprenderemos a:

- Implementar el registro de información en tablas **cabecera–detalle**.
- Entender por qué el desarrollador debe conocer criterios de implementación en SQL Server.
- Manejar conceptos **transaccionales** dentro de un sitio web con EF Core.
- Crear y descargar archivos Excel con **EPPlus** desde la aplicación.

Del stack antiguo (Web Forms, AJAX Control Toolkit, GridView, EF6/EDMX) pasamos al actual: **ASP.NET Core Razor Pages**, **.NET 10**, **EF Core 10** y un modal de **Bootstrap con AJAX (fetch)**, como en las sesiones anteriores.

**/ DESARROLLO DE APLICACIONES TIPO
CARRITO DE COMPRAS O REGISTRO DE
TABLAS MAESTRO-DETALLE APLICANDO
CRITERIOS TRANSACCIONALES**

/ ¿QUÉ ES UNA TRANSACCIÓN?

- Una **transacción** es una operación formada por **varios pasos**: su éxito depende de que **todos** los pasos se completen correctamente.
- Se ejecuta de forma **atómica**: o se realizan todas las operaciones, o ninguna deja efecto en la base de datos (no quedan estados a medias).
- Se puede implementar desde la **base de datos** (recomendable en un procedimiento almacenado) o desde el **código** de la capa de datos con EF Core.

Las transacciones cumplen las propiedades **ACID**: Atomicidad, Consistencia, Aislamiento y Durabilidad. En esta sesión nos centramos en la **atomicidad**: todo o nada.

/ ¿QUÉ ES UNA TRANSACCIÓN? · EJEMPLOS

- **Filmar una escena de cine:** si una toma no queda bien a criterio del director, se vuelve a filmar toda la escena desde el inicio hasta que quede correcta.
- **Transferencia entre dos cuentas:** descontar de una cuenta y abonar en la otra deben ocurrir juntas. Si un paso falla, se cancela toda la operación y el dinero no desaparece.
- **Registrar una orden de compra:** grabar la cabecera y **todos** sus detalles debe ser una sola unidad. Si falla un detalle, no debe quedar la cabecera sola en la base.

/ CASO DE USO: TABLAS MAESTRO-DETALLE

¿Qué tablas participan al registrar una orden de compra?

- La cabecera **Tb_Orden_Compra** y el detalle **Tb_Detalle_Compra** son los protagonistas; **Tb_Producto** participa como “artista invitada” para elegir los artículos.
- Una orden tiene **una** cabecera y **muchos** detalles (relación uno a muchos).

Tb_Orden_Compra (maestro)

Num_oco (PK)
Fec_oco
Cod_prv (FK)
Fec_ate
Est_oco
Fec_Registro
Usu_Registro

Tb_Detalle_Compra (detalle)

Num_oco (PK, FK)
Cod_pro (PK, FK)
Can_ped

Tb_Producto (invitada)

Cod_pro (PK)
Des_pro
Pre_pro
Stk_act
Est_pro

/ ¿QUÉ NECESITAMOS EN LA APLICACIÓN?

- Con EF Core, las tres tablas deben estar en el **modelo** (clases y `DbSet`), con sus **llaves foráneas** configuradas en el `DbContext`.
- En la **capa de datos** creamos los métodos: consultas con **LINQ** o llamadas a **procedimientos almacenados**. La página nunca toca la base directamente (regla de oro).
- El formulario debe ser ágil. En lugar del antiguo **ModalPopup de AJAX Toolkit** usamos un **modal de Bootstrap** y **AJAX con fetch** para agregar los detalles sin recargar.

Equivalencias con el curso antiguo: **ModalPopup** → **modal de Bootstrap**, **GridView + DataTable en ViewState** → **tabla HTML + arreglo en JavaScript**, **EF6/EDMX** → **EF Core 10 (database-first, clases manuales)**.

/ CAPA DE PRESENTACIÓN: FORMULARIO DE ORDEN DE COMPRA

Flujo del registro (estilo carrito de compra):

1. **Selecciona el proveedor** en un combo. Los productos del modal se filtran por ese proveedor.
2. **Modal de Bootstrap** para agregar un detalle: elegir producto y cantidad.
3. Cada detalle se agrega a un **arreglo en memoria (JavaScript)** y se muestra en una **tabla HTML**.
4. Los detalles se pueden **quitar** de la tabla mientras se arma la orden.
5. Al **Grabar**, se envía toda la orden (cabecera + detalles) por **fetch** y se guarda en la base en **una sola transacción**.

El “carrito” vive en el **navegador** mientras se arma. Solo cuando el usuario confirma se hace **un POST** con todo el pedido. Así se evita grabar órdenes incompletas.

/ ¿CÓMO SE GARANTIZA EL “TODO O NADA”?

Tres formas, de la más simple a la más controlada:

1) Un solo SaveChanges (transacción implícita de EF Core)

```
// EF Core agrupa los cambios en una transaccion implicita
_ctx.OrganesCompra.Add(orden);    // cabecera + sus detalles
await _ctx.SaveChangesAsync();    // se graba TODO o NADA
```

2) Transacción explícita en el código

```
using var tx = await
_ctx.Database.BeginTransactionAsync();
try {
    // ... varias operaciones sobre el contexto
    await _ctx.SaveChangesAsync();
    await tx.CommitAsync();    //
confirma
} catch {
    await tx.RollbackAsync();    //
deshace todo
    throw;
}
```

3) Transacción en un procedimiento almacenado

```
CREATE PROCEDURE usp_RegistrarOrdenCompra ...
AS
BEGIN
    BEGIN TRY
        BEGIN TRAN;
        INSERT INTO Tb_Orden_Compra ...; --
cabecera
        INSERT INTO Tb_Detalle_Compra ...; -- cada
detalle
        COMMIT;
    END TRY
    BEGIN CATCH
        ROLLBACK; THROW;
    END CATCH
END
```

/ LABORATORIO

/ LABORATORIO 1

Registrar una orden de compra (maestro–detalle) con capa de presentación web

- Implementaremos el registro de una tabla cabecera–detalle en **ASP.NET Core Razor Pages**, con **modal de Bootstrap** y **AJAX**, emulando un carrito de compra.
- El guardado se hará de forma **transaccional**: la cabecera y todos los detalles se graban como una sola unidad.

Base de datos: **VentasLeon** (continuidad del curso), con las tablas Tb_Orden_Compra, Tb_Detalle_Compra y Tb_Producto.

**/ GENERAR REPORTES EXCEL DESDE ASP.NET
CORE Y EPPLUS**

/ GENERAR REPORTES EXCEL CON EPPLUS

Recordando...

- La información es uno de los recursos más valiosos de un sistema. Para tomar buenas decisiones, los usuarios necesitan **consultas efectivas** y poder **exportarlas**.
- Una funcionalidad muy frecuente es **descargar en Excel** el resultado de una consulta o listado. Para eso usamos **EPPlus**, que ya conocimos en la primera parte del curso.

En .NET / EF Core, EPPlus 5+ exige indicar el contexto de licencia una vez al inicio: `ExcelPackage.LicenseContext = LicenseContext.NonCommercial;` (uso educativo). Una alternativa libre es **ClosedXML**.

/ DESCARGAR UN LISTADO EXCEL DESDE LA CONSULTA

- Tomamos como ejemplo la **facturación de un cliente entre dos fechas** (consulta que ya hicimos en la Sesión 13) y le agregamos un botón **Descargar Excel**.
- En Razor Pages, un handler arma el archivo con EPPlus y lo devuelve como **File** para que el navegador lo descargue.

Handler que genera y devuelve el Excel

```
ExcelPackage.LicenseContext = LicenseContext.NonCommercial;

using var paquete = new ExcelPackage();
var hoja = paquete.Workbook.Worksheets.Add("Facturas");
hoja.Cells["A1"].Value = "LISTADO DE FACTURAS POR CLIENTE";
// ... escribir encabezados y filas de la consulta ...

return File(paquete.GetAsByteArray(),
    "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
    "Facturas.xlsx");
```

/ CONTENIDO DEL EXCEL DESCARGADO

- El archivo generado por EPPlus incluye un **encabezado con la marca ISIL**, la **razón social** del cliente consultado y el **rango de fechas** del reporte.
- Debajo se listan las facturas con su número, fechas, total, vendedor y estado, y un total de registros al pie.

ISIL/ LISTADO DE FACTURAS — CLIENTE / FECHAS					
Nro. Factura	Fec. Factura	Fec. Cancelación	Total (S/)	Vendedor	Estado
FA1223	09/10/2020	—	440.00	Zevallos, Vilma	Pendiente
FA1604	20/01/2021	15/02/2021	16,784.24	Sánchez, Alexis	Cancelada
FA1711	17/12/2020	—	2,263.92	Soto, Javier	Pendiente
Total registros: 3					

/ LABORATORIO

/ LABORATORIO 2

Listados Excel sobre una consulta de valor de negocio, con capa de presentación web

- Generaremos un archivo Excel con EPPlus sobre el resultado de una **consulta de valor de negocio** dentro de una aplicación **ASP.NET Core**.
- El reporte se descargará directamente desde la página, con formato y encabezado ISIL.

/ CONCLUSIONES

- Podemos implementar procesos **transaccionales** en sitios web con EF Core, ya sea con un solo `SaveChanges`, una transacción explícita o un procedimiento almacenado.
- Con esto estamos en condiciones de construir escenarios como **matrículas, reservas o pedidos**, al estilo de los “carritos de compra” de los sitios reales.
- Las **descargas de Excel** con EPPlus agregan valor a la aplicación al entregar la información en un formato que el usuario puede analizar y compartir.

/ BIBLIOGRAFÍA

Páginas web y otros recursos

- **Microsoft Learn:** “Using Transactions” (EF Core). learn.microsoft.com/ef/core/saving/transactions
- **Microsoft Learn:** “Save data / SaveChanges” (EF Core). learn.microsoft.com/ef/core/saving/
- **EPPlus Software:** documentación oficial y contexto de licencia. epplussoftware.com
- **ClosedXML (alternativa libre):** github.com/ClosedXML/ClosedXML

Fecha de consulta: julio de 2026. Referencias actualizadas al stack ASP.NET Core / EF Core.